Express.js Response Methods –

1. Introduction to Server Responses in Express.js

Routes are defined using HTTP methods such as:

```
app.get('/route', (req, res) => {
  // response logic
});
```

The **callback function** receives:

- `req` → request object
- `res` → response object

This tutorial focuses on **methods available on the `res` (response) object** in Express.js.

---

2. `res.send()` – Sending Responses

Purpose

Sends a response to the client. It is the most commonly used response method.

Supported Data Types

`res.send()` can send:

- Text
- HTML
- JavaScript Objects
- Arrays
- Buffers (used in audio/video streaming)

Examples

Sending Text

```
res.send("Hello World");
```

Sending HTML

```
res.send("<h1>Home Page</h1>");
```

Sending an Object (Auto-converted to JSON)

```
res.send({ name: "Aman", age: 25 });
```

Sending an Array

```
res.send(["Apple", "Banana", "Mango"]);
```

Important: When objects or arrays are passed, Express automatically converts them to **JSON format**.

---

3. `res.json()` – Sending JSON Only

Purpose

Used when the server must send **JSON responses only**, especially for APIs.

Example

```
res.json({ name: "Rahul", age: 30 });
```

Array of Objects

```
const users = [
  { id: 1, name: "Peter Parker" },
  { id: 2, name: "Leon S Kennedy" }
];

res.json(users);
```

This method explicitly returns JSON and is preferred for **REST APIs**.

---

4. `res.jsonp()` – JSON with Padding (JSONP)

Purpose

Used for **cross-domain requests** (older technique).

Key Points

- Returns JSON wrapped inside a callback function
- Considered **less secure**

- Requires CORS handling in real projects

Example

```
res.jsonp({ name: "User", age: 20 });
```

With a callback:

```
/api?callback=myFunction
```

Response:

```
myFunction({ name: "User", age: 20 });
```

Recommendation

Use `res.json()` with **CORS middleware** instead of JSONP.

---

5. `res.redirect()` – Redirecting Users

Purpose

Redirects the user from one route to another route or website.

Example: Redirect to Another Route

```
res.redirect('/users');
```

Example: Redirect to External Website

```
res.redirect('https://google.com');
```

Redirect Status Codes

Code	Meaning
301	Permanent redirect
302	Temporary redirect
303	Redirect after POST/PUT

Code	Meaning
307	Temporary redirect (non-GET)
308	Permanent (non-GET)

Example:

```
res.redirect(301, '/new-page');
```

Redirect Back

```
res.redirect('back');
// OR
res.redirect('..');
```

---

6. `res.render()` – Rendering HTML with Template Engines

Purpose

Used to render **dynamic HTML pages** using a **template engine**.

Template Engine Used

EJS (Embedded JavaScript)

Installation

```
npm install ejs
```

Setup

```
app.set('view engine', 'ejs');
```

Folder Structure

```
project/
 ├─ views/
 │   └─ user.ejs
 └─ app.js
```

Route Example

```
app.get('/user', (req, res) => {
  res.render('user');
});
```

Note:

- `.ejs` extension is NOT written in `res.render()`
- Express automatically looks inside the `views` folder

---

7. `res.download()` – Force File Download

Purpose

Forces the browser to **download a file**.

Example

```
res.download('./files/shortcut.pdf');
```

Custom Download Name

```
res.download('./files/shortcut.pdf', 'document.pdf');
```

Supported file types:

- PDF
- Images
- Audio
- Video
- ZIP, etc.

---

8. `res.sendFile()` – Open File in Browser

Purpose

Displays files directly in the browser instead of forcing download.

Example (Relative Path)

```
res.sendFile('./files/shortcut.pdf');
```

Example (Absolute Path)

```
res.sendFile(__dirname + '/files/shortcut.pdf');
```

Difference Between `download` and `sendFile`

Method	Behavior
`download()`	Forces Save dialog
`sendFile()`	Opens file in browser tab

---

9. `res.end()` – End the Response

Purpose

Stops the response immediately.

Example

```
res.write("Processing...");
res.end();
```

Used when:

- Ending response conditionally
- No further output is needed

---

10. `res.sendStatus()` – Send HTTP Status Only

Purpose

Sends **only a status code** as the response.

Example

```
res.sendStatus(404);
```

Browser Output:

```
Not Found
```

11. `res.status()` – Send Status with Data

Purpose

Attach a status code to a response.

Example

```
res.status(200).send("Hello");
```

Common HTTP Status Codes

Code	Meaning
200	OK
201	Created
403	Forbidden
404	Not Found
500	Internal Server Error
503	Service Unavailable
504	Gateway Timeout

12. `res.headersSent` – Check If Response Is Sent

Purpose

Checks whether the response has already been sent.

Example

```
console.log(res.headersSent); // false
res.send("Hello");
console.log(res.headersSent); // true
```

Useful to prevent **multiple responses** in complex logic.

13. `res.set()` and `res.get()` – Custom Headers

Purpose

Acts like **variables** stored in response headers.

Set Header

```
res.set('custom-header', 'hello123');
```

Get Header

```
console.log(res.get('custom-header'));
```

Example

```
res.set('custom-header', 'hello123');
res.send('Header Set');
```

Used for:

- Custom metadata
- Tokens
- API headers

---

14. Summary of Response Methods

Method	Use Case
`send()`	Send any type of data
`json()`	Send JSON APIs
`jsonp()`	Legacy cross-domain
`redirect()`	Redirect users
`render()`	Render HTML pages
`download()`	Force file download
`sendFile()`	Open file in browser
`end()`	End response
`sendStatus()`	Status only
`status()`	Status + data
`headersSent`	Check response state
`set()` / `get()`	Manage headers

This tutorial establishes **complete mastery of Express.js response handling**, which is essential for:

- APIs
- File serving
- Authentication
- Dynamic websites
